IA como servicio (AIaaS)

Integración, implementación, uso y costes

Proyecto didáctico para IFCD107

Material técnico de apoyo

23 de octubre de 2026

Resumen

Este proyecto presenta una guía práctica para consumir algoritmos de inteligencia artificial mediante servicios gestionados y API. Compara opciones de nube, explica una integración REST segura y describe un flujo reproducible usando como caso de estudio el servicio de regresión KNN del proyecto IFCD107: un CSV sintético de 5 000 filas, un endpoint con validación K-Fold y otro con Leave-One-Out. Incluye ejemplos de petición, arquitectura, despliegue, criterios de selección, una estimación transparente de costes y repositorios públicos de referencia.

Índice

1. Objetivo y alcance **2**

2. Plataformas y criterios de integración **2**
2.1. Criterios de decisión 2

3. Arquitectura técnica de referencia **3**

4. Contrato de la API didáctica **3**
4.1. Autenticación 3

5. Preparación del ejemplo KNN **3**
5.1. Flujo K-Fold 4
5.2. Flujo Leave-One-Out 4

6. Implementación y ejecución **4**
6.1. Preparar entorno 4
6.2. Crear token 4
6.3. Enviar CSV con K-Fold 5
6.4. Enviar CSV con LOO 5
6.5. Interpretar y conservar resultados 5

7. Despliegue de una implementación en nube **5**

8. Costes y ejemplo de presupuesto **6**
8.1. Escenarios orientativos 7

9. Repositorios públicos y ejemplos **7**

10. Seguridad, privacidad y operación **7**

11. Conclusiones **8**

1. Objetivo y alcance

AIaaS (*Artificial Intelligence as a Service*) permite consumir modelos, algoritmos o infraestructura de IA por API, SDK o consola web, sin que el usuario tenga que mantener toda la infraestructura de ejecución. El proveedor puede aportar modelos listos para usar, endpoints gestionados, entrenamiento, almacenamiento, escalado, monitorización o solo capacidad de cómputo.

El término cubre productos distintos. Una API de un modelo fundacional no equivale a desplegar un estimador clásico como K-Nearest Neighbors (KNN). En este material se distinguen:

- **Inferencia de modelos preentrenados**: se envían datos o texto a un modelo ya publicado y se recibe una respuesta.
- **Entrenamiento y despliegue gestionados**: se registra o entrena un modelo propio y se publica un endpoint para predicciones.
- **Infraestructura gestionada**: se elige un modelo y una instancia; el proveedor opera el servidor y el escalado.

El ejemplo de IFCD107 usa regresión KNN supervisada. No usa un LLM para calcular las predicciones.

2. Plataformas y criterios de integración

Plataforma	**Uso típico**	**Consideración para IFCD107**
Amazon SageMaker AI	Entrenamiento y alojamiento de modelos propios; inferencia en tiempo real, serverless, asíncrona y por lotes.	Adecuado para desplegar un pipeline scikit-learn con controles de IAM. Se factura por recursos utilizados y modalidad.
Google Cloud Vertex AI	Entrenamiento AutoML o personalizado, registro de modelos y predicciones online/batch.	Alternativa de ciclo de vida gestionado; desplegar un contenedor o modelo compatible y controlar región y escalado.
Microsoft Azure Machine Learning / Microsoft Foundry	Endpoints online de modelos propios y catálogo de modelos/servicios de IA.	Azure Machine Learning encaja con modelos tabulares propios; Foundry facilita acceso a modelos fundacionales y agentes.
Hugging Face Inference Endpoints / Providers	Endpoints dedicados para alojar modelos del Hub y llamadas API serverless a modelos compatibles.	Sencillo para explorar modelos abiertos; para KNN clásico puede requerirse un contenedor personalizado y verificar compatibilidad.

2.1. Criterios de decisión

Antes de escoger, comparar: tipo de modelo y formato admitido, ubicación de datos, autenticación e identidad, región, latencia, volumen y tamaño de solicitudes, escalado a cero, cuota, observabilidad, retención de logs, licencia, SLA y precio total. Para un proyecto académico de regresión tabular, comenzar localmente permite aprender el algoritmo sin costes de nube. Publicar el endpoint es una etapa adicional.

3. Arquitectura técnica de referencia

```
Cliente (Postman / aplicación Python)
        | HTTPS + token Bearer + CSV multipart
        v
API Flask del proyecto IFCD107
        | valida identidad, tamaño y columnas
        v
Pipeline scikit-learn: StandardScaler + KNeighborsRegressor
        | KFold o LeaveOneOut + cross_val_predict
        v
Métricas OOF + predicciones + modelo final + log + gráfico
```

La demostración local persiste los resultados en directorios de ejecuciones. En una nube, la misma lógica se puede empaquetar en un contenedor y desplegar detrás de un endpoint gestionado. La API de aplicación puede actuar como capa intermedia: autentica al usuario, valida el contrato y llama al endpoint de modelo. Conviene separar credenciales del proveedor y del cliente final.

4. Contrato de la API didáctica

La API Flask del proyecto del curso ofrece dos operaciones multipart:

- `POST /api/v1/regresion/knn/kfold`: K-Fold con 5 pliegues predeterminados.
- `POST /api/v1/regresion/knn/leave-one-out`: Leave-One-Out (LOO), que deja una fila fuera en cada iteración.

Ambas requieren `Authorization: Bearer <token>` y campos de formulario: `reference` (identificador de ejecución), `target` (por defecto `y`), `n_neighbors` (por defecto 5), `random_state` (por defecto 42), `cv` (por defecto 5, relevante para K-Fold) y `file` (CSV). Para este ejemplo los predictores y la variable objetivo deben ser numéricos, sin valores ausentes o infinitos. El límite de la ruta es 4 MiB y el conjunto está limitado a 5 000 filas.

La respuesta JSON comunica el identificador, técnica de validación, número de pliegues, columnas, parámetros, métricas out-of-fold (R^2, RMSE y MAE) y rutas relativas a los artefactos guardados. Las métricas OOF corresponden a predicciones sobre filas que no participaron en el ajuste de su pliegue; el modelo guardado al final se vuelve a ajustar con todas las filas para usarlo en inferencia futura.

4.1. Autenticación

Primero se solicita un token a `POST /api/v1/auth/token`, enviando las credenciales configuradas en el entorno. La API devuelve un token temporal. Se debe guardar en una variable de entorno, no escribir una clave real dentro del código ni subirla al repositorio. En despliegues con varias instancias, la clave de firma debe ser persistente y estar en un gestor de secretos. HTTPS es obligatorio fuera de un entorno local.

5. Preparación del ejemplo KNN

El generador `scripts/generate_knn_example.py` crea `ejemplos/datos_knn_5000.csv` con semilla fija. Sus columnas son:

- `x1`, `x2`, `x3`, `x4`: variables predictoras numéricas en escalas diferentes.
- `y`: objetivo sintético calculado con una relación no lineal y ruido aleatorio.

Los registros no son observaciones reales y solo sirven para aprender. El pipeline aplica `StandardScaler` antes de `KNeighborsRegressor`, ya que KNN usa distancias y una variable con rango grande podría dominar a las demás. El escalado forma parte del pipeline para que en cada pliegue se ajuste usando únicamente el subconjunto de entrenamiento, reduciendo el riesgo de fuga de información.

5.1. Flujo K-Fold

Se divide el conjunto en k particiones. Para cada iteración, una partición se reserva para validación y las restantes entrenan el pipeline. Con $k = 5$ se producen cinco modelos temporales y una predicción OOF por fila. El barajado usa una semilla fija para hacer repetible el ejemplo. Es un equilibrio habitual entre coste y evaluación; los resultados dependen de la partición, por lo que en proyectos reales también se puede repetir la validación o reservar un conjunto de prueba.

5.2. Flujo Leave-One-Out

Con M filas se ejecutan M iteraciones: una fila para validar y $M - 1$ filas para entrenar. En este ejemplo son 5 000 ajustes. LOO aprovecha casi todos los datos en cada entrenamiento, pero es mucho más costoso y puede tardar considerablemente más. No es una opción automáticamente mejor; conviene usarla cuando el tamaño del conjunto y el coste de entrenamiento lo permitan.

6. Implementación y ejecución

6.1. Preparar entorno

Desde la carpeta del proyecto:

```
python -m venv .venv
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Configurar las variables del entorno según `.env.example` y la guía del repositorio. Usar una contraseña de práctica local; no reutilizar contraseñas personales. Iniciar Flask con el mecanismo indicado en el README del proyecto y comprobar `GET /api/v1/health`.

6.2. Crear token

El siguiente ejemplo muestra el formato. Sustituir usuario y contraseña por los valores configurados localmente:

```
import requests

base = "http://127.0.0.1:5000"
login = requests.post(
    f"{base}/api/v1/auth/token",
    json={"username": "alumno", "password": "CAMBIAR_EN_LOCAL"},
    timeout=20,
)
login.raise_for_status()
token = login.json()["access_token"]
```

No usar HTTP sin TLS en una red pública. Para producción, alojar el secreto en el gestor de secretos de la plataforma y restringir el acceso al endpoint.

6.3. Enviar CSV con K-Fold

```
from pathlib import Path

csv_path = Path("ejemplos/datos_knn_5000.csv")
with csv_path.open("rb") as data:
    response = requests.post(
        f"{base}/api/v1/regresion/knn/kfold",
        headers={"Authorization": f"Bearer {token}"},
        data={
            "reference": "knn_ifcd107_kfold",
            "target": "y",
            "n_neighbors": "5",
            "cv": "5",
            "random_state": "42",
        },
        files={"file": (csv_path.name, data, "text/csv")},
        timeout=300,
    )
response.raise_for_status()
print(response.json())
```

6.4. Enviar CSV con LOO

Cambiar la URL por `/api/v1/regresion/knn/leave-one-out`, mantener el mismo CSV y parámetros KNN y aumentar el timeout del cliente según el tiempo observado. Con 5 000 registros LOO implica 5 000 entrenamientos; ejecutar primero K-Fold para confirmar que el entorno y los datos funcionan. El servidor limita la carga a 5 000 filas para esta demostración.

6.5. Interpretar y conservar resultados

Revisar `metrics_oof`. RMSE está en las unidades de `y`; MAE es el error absoluto medio y R^2 compara el ajuste con una referencia constante. No se debe decidir qué modelo gana con una métrica aislada: importa el objetivo del problema y el comportamiento en datos nuevos. Conservar CSV de entrada, parámetros, versión del código, semilla y artefactos para reproducir y auditar el resultado.

7. Despliegue de una implementación en nube

Flujo recomendado:

1. **Preparar artefactos**: fijar dependencias, validar esquema de entrada, serializar un pipeline completo y definir una función de predicción compatible.
2. **Contenerizar**: crear una imagen Docker reproducible con runtime, bibliotecas y código; incluir una ruta de salud y una ruta de predicción JSON.
3. **Almacenar y versionar**: guardar imagen y modelo en registros privados con permisos mínimos. No incluir datos personales ni credenciales en la imagen.
4. **Crear endpoint**: elegir región, CPU/GPU adecuada, número mínimo/máximo de réplicas, autenticación, red privada o pública y timeout.
5. **Integrar cliente**: enviar JSON validado por HTTPS; para cargas tabulares grandes, valorar almacenamiento de objetos y procesamiento batch en vez de enviar archivos grandes al endpoint online.

6. **Observar y controlar**: medir latencia, errores, uso, coste y cambios de distribución de datos; configurar alertas y límites de gasto.
7. **Actualizar y retirar**: probar una versión candidata, promoverla con control de tráfico y eliminar o pausar recursos que no se usan.

KNN no tiene una fase de entrenamiento costosa como una red neuronal, pero mantiene los datos de entrenamiento o una estructura equivalente para buscar vecinos al inferir. Por eso el tamaño del conjunto, latencia y concurrencia deben medirse. Para una API de KNN pequeña, una instancia CPU suele ser el primer punto de evaluación; la elección final depende de medidas reales.

8. Costes y ejemplo de presupuesto

El coste depende de proveedor, región, tipo de máquina, réplicas, horas activas, entrenamiento, almacenamiento, tráfico de red, logs y servicios auxiliares. Los precios cambian; la tabla siguiente es una referencia de consulta en octubre de 2026, en dólares estadounidenses, antes de impuestos y descuentos. No es una cotización ni un compromiso de tarifa.

Opción	**Modelo de cobro**	**Dato verificable / cálculo**
Hugging Face Inference Endpoint, CPU AWS x1	Por instancia y tiempo en estado ejecutable; cobro por minuto según documentación.	Tarifa publicada de referencia: US$0.033/h. Operación continua: $0.033 \times 730 \approx$ US$24.09/mes por réplica. Almacenamiento y otros conceptos pueden modificar el total.
Hugging Face Inference Providers	Pago por uso según modelo/proveedor y cómputo.	La documentación indica créditos mensuales sujetos al tipo de cuenta y cobro posterior por uso. Consultar precio del modelo y consumo efectivo antes de producción.
Amazon SageMaker AI	Cómputo de entrenamiento, hosting, almacenamiento y procesamiento usados.	No existe un único coste de endpoint: estimar tipo de instancia, región y horas en la calculadora de AWS. Para tráfico irregular, serverless puede evitar pagar recursos ociosos, sujeto a sus límites.
Google Cloud Vertex AI	Cómputo, almacenamiento y herramientas usadas; cambia según región y tipo de máquina.	Para modelos tabulares personalizados, consultar calculadora y tarifa de endpoint/instancia en la región elegida; algunas categorías de AutoML tabular requieren consulta comercial.
Azure Machine Learning	Cómputo subyacente, disco, red y servicios asociados.	Calcular instancia, mínimo de réplicas y horas; detener o escalar el despliegue al terminar prácticas.

8.1. Escenarios orientativos

- **Práctica local**: US$0 de cómputo cloud; se usa el equipo del alumno. Costes de conexión o electricidad no incluidos.
- **Endpoint CPU de demostración siempre encendido**: con la tarifa publicada de Hugging Face de US$0.033/h para una instancia AWS CPU x1, 8 horas costarían aproximadamente US$0.26; 730 horas, US$24.09. Es una aproximación aritmética antes de impuestos, redondeos, almacenamiento u otros servicios.
- **API de modelo fundacional**: el gasto depende de modelo, tokens o unidades de cómputo y volumen. Estimar con las tarifas vigentes del modelo seleccionado; fijar presupuestos y límites.

Para controlar gasto, pausar o borrar endpoints de práctica tras la sesión, seleccionar CPU cuando sea suficiente, limitar réplicas, usar escalado a cero si el producto lo permite, y revisar las métricas de facturación. Un endpoint pausado puede tener otros recursos asociados que sigan generando coste.

9. Repositorios públicos y ejemplos

Los repositorios son puntos de partida técnicos, no garantías de que un notebook siga funcionando sin cambios. Revisar licencia, fecha, rama, versiones, región requerida y gasto antes de ejecutar.

- aws/amazon-sagemaker-examples: notebooks para entrenar, evaluar y desplegar modelos con SageMaker; navegar a `inference/` y ejemplos de scikit-learn.
- GoogleCloudPlatform/vertex-ai-samples: ejemplos de Vertex AI, incluyendo `notebooks/official/custom/` y `prediction/` para entrenamiento y predicción de modelos propios.
- Azure/azureml-examples: ejemplos de Azure Machine Learning. Complementar con el tutorial oficial de endpoint online de scikit-learn enlazado en las referencias.
- huggingface/huggingface_hub: SDK y guías de Inference Providers y gestión de Inference Endpoints.
- scikit-learn/scikit-learn: biblioteca del algoritmo y documentación de KNeighborsRegressor; útil para comprender el modelo antes de publicarlo como API.
- GoogleCloudPlatform/ai-platform-samples: muestras históricas de AI Platform; el propio repositorio dirige a Vertex AI Samples para material nuevo.

Para el ejercicio de este documento, revisar primero el código del proyecto IFCD107 en `regression/knn_regresso` `api_functions.py`, `app.py` y la colección Postman del repositorio.

10. Seguridad, privacidad y operación

- Enviar únicamente las columnas necesarias; anonimizar o sintetizar datos de prácticas.
- Revisar ubicación, tratamiento, conservación y eliminación de datos con el responsable del servicio.
- Usar TLS, tokens de corta duración, rotación, control de acceso mínimo y secretos fuera del repositorio.
- Limitar tamaño y tipo de peticiones; validar esquema, rangos, valores ausentes y tamaño de salida.

- Evitar registrar tokens, contraseñas o payloads sensibles. Registrar identificadores, versión de modelo, parámetros y estado.
- Establecer límites de gasto, alertas, cuota y un procedimiento para apagar despliegues.
- Validar precisión y deriva con datos representativos; la validación cruzada no sustituye la evaluación de impacto ni las pruebas con datos de producción.

11. Conclusiones

AIaaS permite pasar de un algoritmo de aula a un endpoint consumible por otras aplicaciones, pero delega operación a un proveedor y añade costes, seguridad y decisiones de arquitectura. Para IFCD107, el camino didáctico más controlable es generar el CSV sintético, ejecutar K-Fold, inspeccionar predicciones y métricas, y después comparar LOO conociendo su coste computacional. Solo cuando el contrato de datos, la autenticación y la evaluación están claros conviene desplegar el modelo en una nube.

Referencias y precios

[1] Amazon SageMaker AI, opciones de inferencia y facturación: https://docs.aws.amazon.com/sagemaker/latest/dg/inference-cost-optimization.html y https://aws.amazon.com/sagemaker/ai/faqs/.

[2] Vertex AI, precios: https://cloud.google.com/vertex-ai/pricing.

[3] Microsoft Learn, creación de endpoint online: https://learn.microsoft.com/azure/machine-learning/tutorial-1st-experiment-hello-world.

[4] Hugging Face, precios de Inference Endpoints: https://huggingface.co/docs/inference-endpoints/en/pricing.

[5] Hugging Face, costes de Inference Providers: https://huggingface.co/docs/inference-providers/en/pricing.

[6] Google Cloud, repositorio de ejemplos Vertex AI: https://github.com/GoogleCloudPlatform/vertex-ai-samples.

[7] AWS, repositorio de ejemplos SageMaker: https://github.com/aws/amazon-sagemaker-examples.

[8] Documentación de scikit-learn, KNeighborsRegressor: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsRegressor.html.

Nota de vigencia: tarifas, nombres de producto, cuotas y disponibilidad por región pueden cambiar. Verificar la página de precios y la calculadora oficial justo antes de desplegar. La fecha de consulta indicada para este material es 23 de octubre de 2026.